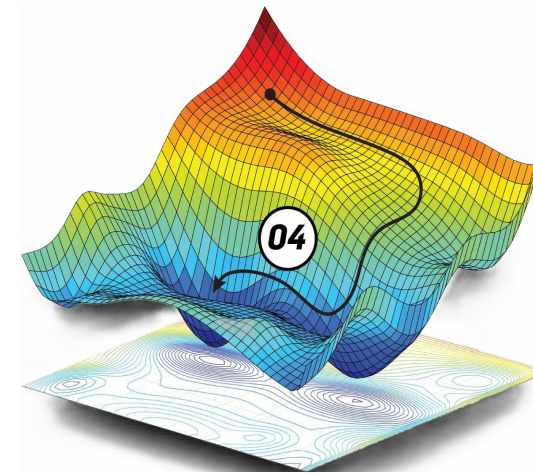




Gradient Descent

Ruhid Ghosh (ID: 2023100000230)
Sadia Alam (ID: 2022200000179)
Southeast University



Introduction

Gradient Descent is an optimization algorithm used to minimize a function, usually a cost function in machine learning or deep learning.

Key Application: Sales Prediction, Linear Regression, Neural Networks.

Problem Statement & The Initial Approach: A company wants to predict its sales based on advertising expenditure. However, the initial model produces inaccurate predictions because its parameters (weights) are not optimized. Therefore, an optimization method is needed to iteratively adjust the model parameters and minimize the prediction error.

Key Idea: Iterative weight optimization.

Improving Accuracy: Learning Rate & Iterative Updates.

The Gradient Descent Algorithm base

Gradient Descent is based on moving in the opposite direction of the gradient to minimize the loss function. The gradient indicates the direction of the steepest increase in loss. Therefore, moving in the opposite direction reduces the loss.

Loss Function (MSE):

$$J(w) = \frac{1}{m} \sum_{i=1}^m (h(x_i) - y_i)^2$$

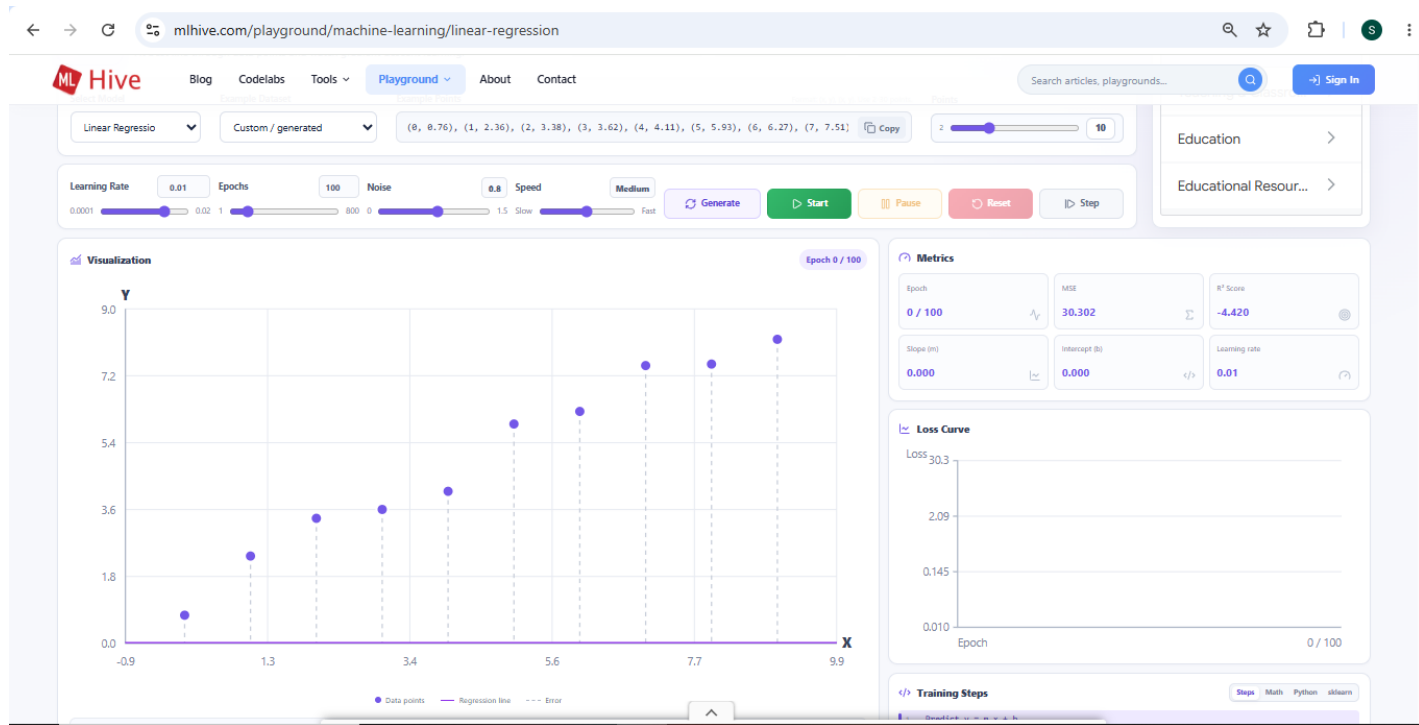
Here: $J(w)$ = cost/loss function; m = total training examples; $h(x_i)$ = model prediction; $h(x_i) - y_i$ = prediction error.

Weight Update Rule:

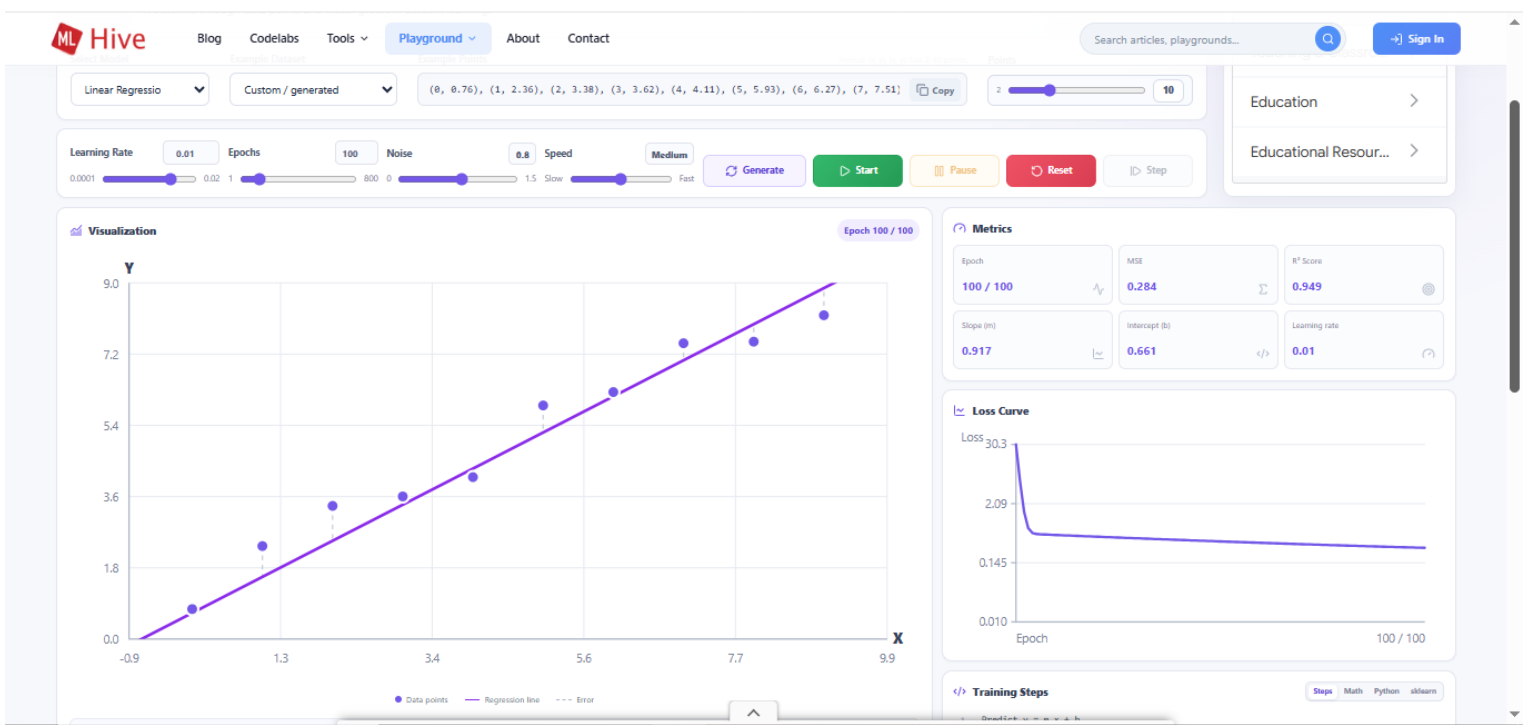
$$w_{new} = w_{old} - \alpha \frac{\partial J(w)}{\partial w}$$

Gradient Descent Visualization

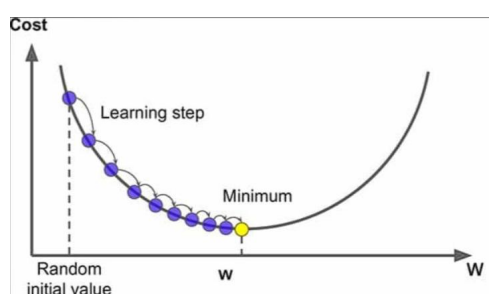
The gradient indicates the direction of the steepest increase in loss. Gradient Descent moves in the opposite direction and repeatedly updates the parameters until the loss is minimized.



Initial State — Before Optimization



Optimized State — After Gradient Descent



Reference: Linear Regression Visualizer - ML Hive Playground

Algorithmic Execution Process

Step 1: Initialize Parameters — Set initial weights w , bias b , and learning rate α .

Step 2: Make Prediction — Calculate $h(x) = wx + b$.

Step 3: Calculate Loss — Measure the prediction error using the Cost/Loss Function.

$$J(w) = \frac{1}{m} \sum_{i=1}^m (h(x_i) - y_i)^2$$

Step 4: Compute Gradient — Calculate the gradient of the loss with respect to the weight:

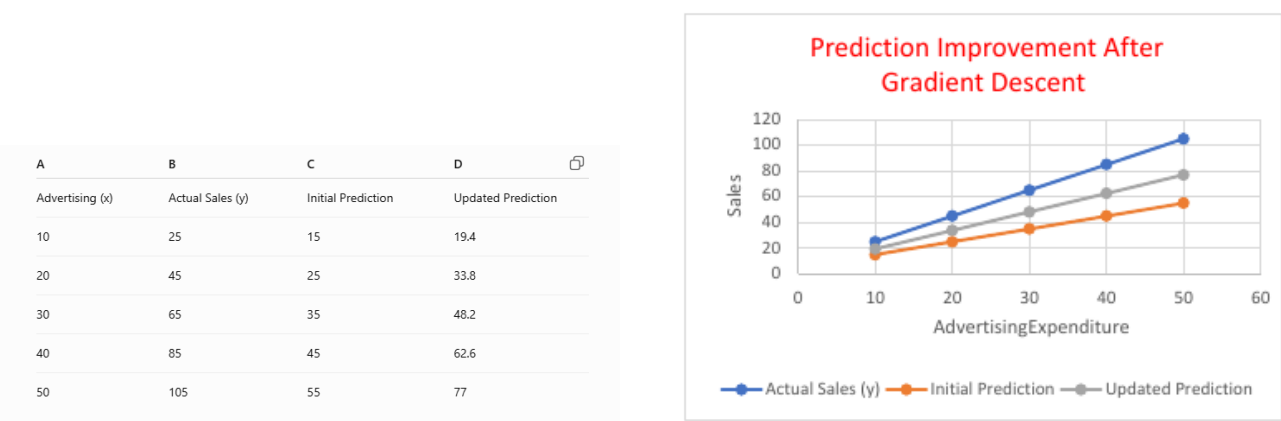
$$\frac{\partial J(w)}{\partial w}$$

Step 5: Update Parameters — Adjust the weight to reduce the loss:

$$w_{new} = w_{old} - \alpha \frac{\partial J(w)}{\partial w}$$

Step 6: Repeat Until Convergence — Repeat prediction, loss calculation, gradient computation, and parameter update until the loss is minimized.

Graph Shows



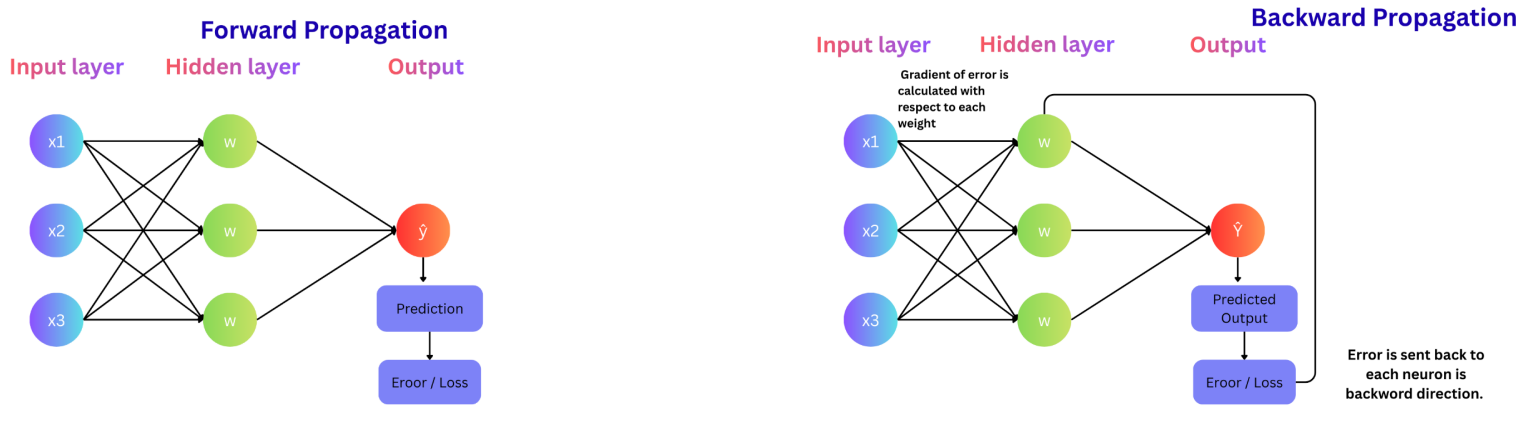
Graph Shows

As advertising expenditure increases, sales also increase. Initially, the model's predictions are far from the actual sales values. After the Gradient Descent weight update, the prediction line moves closer to the actual sales points. This reduces prediction error (loss) and improves model accuracy.

Properties of Gradient Descent

- Iterative Optimization** — Updates model parameters repeatedly to minimize the loss function.
- Gradient-Based** — Uses the gradient to determine the direction of parameter updates.
- Learning Rate Controlled** — Controls the step size of each update.
- Convergence** — Gradually moves toward a minimum of the loss function.
- Scalable** — Efficiently handles models with a large number of parameters.
- Loss Minimization** — Aims to reduce prediction error and improve model performance.

Forward Propagation and Backward Propagation



Forward Propagation passes input data through the network to generate a prediction.

Backward Propagation propagates the error backward through the network to calculate gradients and adjust the weights for improved predictions.

Time and Space Complexity

Let n = number of training samples, d = number of features, and I = number of iterations.

Each iteration requires approximately $O(n \times d)$ operations to calculate the gradient.

Overall Time Complexity:

$$O(I \times n \times d)$$

Space Complexity: $O(n \times d)$ including training data.

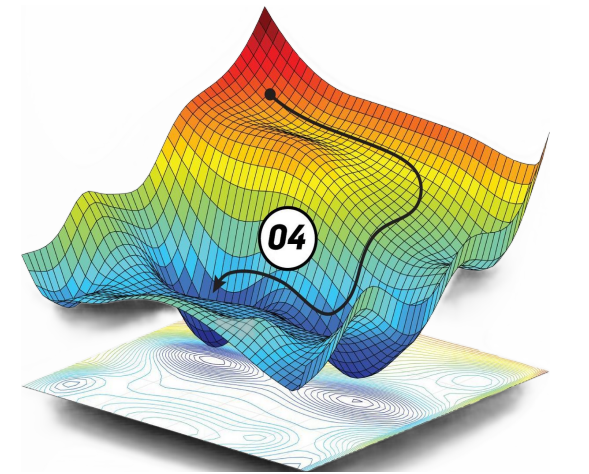
Without training data: $O(d)$.

Gradient Descent reduces the cost function through repeated, gradient-based parameter updates.



Stochastic Gradient Descent (SGD)

Ruhid Ghosh (ID: 2023100000230)
Sadia Alam (ID: 2022200000179)
Southeast University



Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) is an optimization algorithm used to minimize a function, usually a cost function in machine learning or deep learning.

Key Application: Sales Prediction, Linear Regression, Neural Networks.

Problem Statement & The Initial Approach: A company wants to predict its sales based on advertising expenditure. However, the initial model produces inaccurate predictions because its parameters (weights) are not optimized. Therefore, an efficient optimization method is needed to iteratively adjust the model parameters and minimize prediction error.

Key Idea: Iterative weight optimization using individual training examples.

Improving Accuracy: Learning Rate & Frequent Weight Updates.

The Stochastic Gradient Descent Algorithm

SGD updates the model weights using one randomly selected training example at a time instead of the entire dataset. For each example, SGD calculates prediction error, computes the gradient, and immediately updates the weight. This makes optimization faster and more frequent, especially for large datasets.

Weight Update Rule

Loss Function (for one example):

$$J_i(w) = \frac{1}{2} (h(x_i) - y_i)^2$$

Here: $J_i(w)$ = loss for an individual training sample; $h(x_i)$ = model prediction; $h(x_i) - y_i$ = prediction error.

Weight Update Rule:

$$w_{new} = w_{old} - \alpha \frac{\partial J_i(w)}{\partial w}$$

Here: α = learning rate; $J_i(w)$ = loss for the individual training sample; $\partial J_i(w)/\partial w$ = gradient of the individual loss with respect to the weight.

One example → prediction → loss → gradient → immediate weight update

Algorithmic Execution Process — SGD

Step 1: Initialize Parameters — Set initial weights w , bias b , and learning rate α .

Step 2: Select One Training Example — Select one training example (x_i, y_i) from the dataset.

Step 3: Make Prediction — Calculate $h(x_i) = wx_i + b$.

Step 4: Calculate Loss — Measure prediction error:

$$J_i(w) = \frac{1}{2} (h(x_i) - y_i)^2$$

Step 5: Compute Gradient — Calculate $\partial J_i(w)/\partial w$.

Step 6: Update Parameters — Immediately adjust the weight:

$$w_{new} = w_{old} - \alpha \frac{\partial J_i(w)}{\partial w}$$

Step 7: Repeat for the Next Example — Select the next example and repeat prediction, loss calculation, gradient computation, and weight update.

Step 8: Repeat Until Convergence — Continue for multiple epochs until the loss is minimized or the model converges.

Properties of SGD

- **Iterative Optimization** — Updates parameters repeatedly using individual training examples.
- **Stochastic / Sample-Based** — Computes the gradient using one training example at a time.
- **Gradient-Based** — Uses the gradient to determine the direction of updates.
- **Learning Rate Controlled** — Controls the step size of each weight update.
- **Frequent Weight Updates** — Updates parameters after each individual example.
- **Faster Computation** — Requires less computation per update and is efficient for large datasets.
- **Noisy Updates** — Single-example updates can make the optimization path fluctuate.
- **Loss Minimization** — Aims to reduce prediction error through repeated updates.

Real world Application

Problem statement: This report presents a mathematical and algorithmic analysis of fitting a simple linear regression model to predict sales from advertising expenditure. A dataset of five observation pairs is used to evaluate and compare Batch Gradient Descent (BGD) and Stochastic Gradient Descent (SGD).

The analysis tracks w (weight) and b (bias) over five training epochs with initial $w = 1$, $b = 5$, and learning rate $\alpha = 0.0002$. The reported results compare convergence behavior for the small, structured dataset.

Input variables: Advertising Expenditure → Sales

Optimization methods: Batch Gradient Descent (BGD) and Stochastic Gradient Descent (SGD).

Table 1: Advertising Expenditure vs. Actual Sales Training Data

Sample Index (i)	Advertising Expenditure (x_i)	Actual Sales (y_i)
1	10	25
2	20	45
3	30	65
4	40	85
5	50	105

Case study

2.2 Detailed Derivation for Epoch 1

Using initial values $w^{(0)} = 1$, $b^{(0)} = 5$:

1. Predictions & Errors:

$$\begin{aligned} \hat{y}_1 &= 1(10) + 5 = 15, & e_1 &= 15 - 25 = -10 \\ \hat{y}_2 &= 1(20) + 5 = 25, & e_2 &= 25 - 45 = -20 \\ \hat{y}_3 &= 1(30) + 5 = 35, & e_3 &= 35 - 65 = -30 \\ \hat{y}_4 &= 1(40) + 5 = 45, & e_4 &= 45 - 85 = -40 \\ \hat{y}_5 &= 1(50) + 5 = 55, & e_5 &= 55 - 105 = -50 \end{aligned}$$

2. Gradient Computation:

$$\begin{aligned} \frac{\partial J}{\partial w} &= \frac{1}{5} [-10(10) + (-20)(20) + (-30)(30) + (-40)(40) + (-50)(50)] = \frac{-5500}{5} = -1100 \\ \frac{\partial J}{\partial b} &= \frac{1}{5} [-10 - 20 - 30 - 40 - 50] = \frac{-150}{5} = -30 \end{aligned}$$

3. Parameter Updates:

$$\begin{aligned} w^{(1)} &= 1.0 - 0.0002 \times (-1100) = 1.0 + 0.22 = 1.220000 \\ b^{(1)} &= 5.0 - 0.0002 \times (-30) = 5.0 + 0.006 = 5.000600 \end{aligned}$$

2

3 Stochastic Gradient Descent (SGD)

In Stochastic Gradient Descent, parameters are updated iteratively after evaluating each individual sample i :

$$w \leftarrow w - \alpha(b - y_i)x_i, \quad b \leftarrow b - \alpha(b - y_i) \quad (5)$$

3.1 Step-by-Step Derivation for Epoch 1

Starting with $w = 1.000000$, $b = 5.000000$:

• **Sample 1** ($x = 10, y = 25$):

$$\hat{y}_1 = 1(10) + 5 = 15.0 \Rightarrow e_1 = -10.0$$

$$w = 1.0 - 0.0002(-10 \times 10) = 1.020000, \quad b = 5.0 - 0.0002(-10) = 5.002000$$

• **Sample 2** ($x = 20, y = 45$):

$$\hat{y}_2 = 1.02(20) + 5.002 = 25.4020 \Rightarrow e_2 = 25.402 - 45 = -19.5980$$

$$w = 1.02 - 0.0002(-19.598 \times 20) = 1.039592, \quad b = 5.002 - 0.0002(-19.598) = 5.003920$$

• **Sample 3** ($x = 30, y = 65$):

$$\hat{y}_3 = 1.039592(30) + 5.00392 = 37.95768 \Rightarrow e_3 = 37.95768 - 65 = -27.04232$$

$$w = 1.039592 - 0.0002(-27.04232 \times 30) = 1.260946, \quad b = 5.00392 - 0.0002(-27.04232) = 5.011328$$

• **Sample 4** ($x = 40, y = 85$):

$$\hat{y}_4 = 1.260946(40) + 5.011328 = 55.637108 \Rightarrow e_4 = 55.637108 - 85 = -29.362892$$

$$w = 1.260946 - 0.0002(-29.362892 \times 40) = 1.407140, \quad b = 5.011328 - 0.0002(-29.362892) = 5.017241$$

• **Sample 5** ($x = 50, y = 105$):

$$\hat{y}_5 = 1.407140(50) + 5.017241 = 75.357201 \Rightarrow e_5 = 75.357201 - 105 = -29.642799$$

$$w = 1.407140 - 0.0002(-29.642799 \times 50) = 1.748402, \quad b = 5.017241 - 0.0002(-29.642799) = 5.022266$$

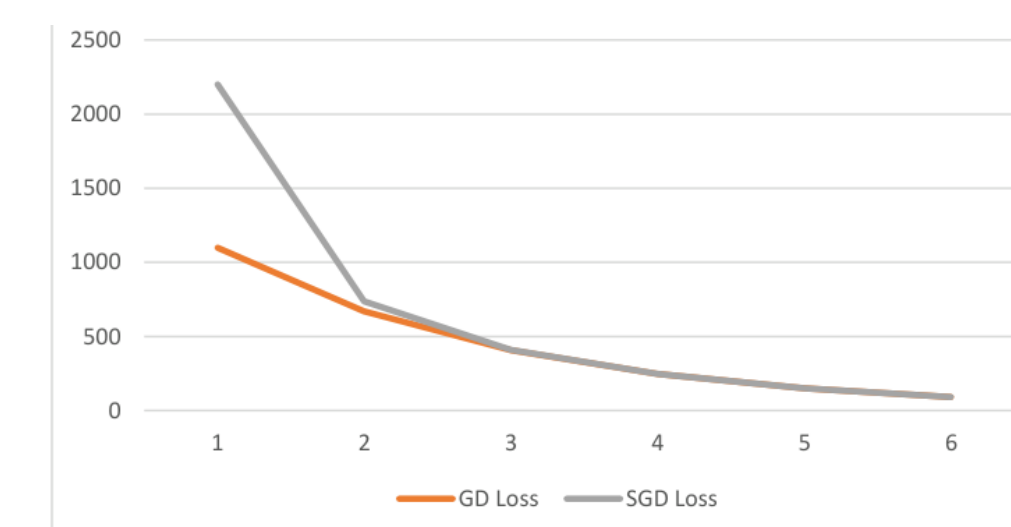
Table 2: Batch Gradient Descent Progress Across 5 Epochs

Epoch	Wtrain	Wtest	Wtrain	Wtest	Wtrain	Wtest
1	1.000000	5.000000	-1100.0000	-30.0000	1.220000	5.000600
2	1.220000	5.000600	-827.6200	-23.3840	1.391564	5.010679
3	1.391564	5.010679	-648.9592	-18.2424	1.525356	5.014327
4	1.525356	5.014327	-521.6787	-14.2250	1.626962	5.017172
5	1.626962	5.017172	-406.8241	-11.0921	1.711056	5.019391

The case study records parameter updates for Gradient Descent and Stochastic Gradient Descent. The source report uses the five-observation sales dataset and tracks w and b across training epochs.

Comparison with Another Algorithm

GD vs SGD Loss Graph



GD vs. SGD Loss Convergence

In this case study, the loss reduction of Gradient Descent (GD) and Stochastic Gradient Descent (SGD) is compared for six recorded epochs. The GD loss decreases from 1100 to 91.5017, while the SGD loss decreases from 1100 to 0.001. The graph shows the difference in loss reduction and convergence behavior between GD and SGD.

References

1. **Gradient Descent intuition**
It explains the intuition behind Gradient Descent, optimization, and its application to Linear Regression.
Stanford CS168 — Gradient Descent Notes
2. **Gradient Descent in Linear Regression**
Linear Regression, MSE/Cost Function, Learning Rate, Gradient Calculation, Parameter Update, Iteration/Convergence.
GeeksforGeeks — Gradient Descent in Linear Regression
3. **Stochastic Gradient Descent (SGD)**
Single training example, parameter update, learning rate, gradient calculation, iterative process, faster updates, noisy/fluctuating updates.
GeeksforGeeks — Stochastic Gradient Descent (SGD)
4. **Book reference**
The concept of Gradient Descent is based on Nafis Neelal's *Machine Learning Algorithms* (2018), p. 47.
Neelal, N. (2018). *Machine Learning Algorithms*. 5th print, January 2022, p. 47.